

Rapport final

Système interactif pour le bien-être des Aras



Membres du groupe

BEN SRIBIT Haytam

BOUNOUA Ilyass

Dia Mohamadou

EL HOUDI Anass

RODRIGUEZ Triana Santiago Alexander

VAZEILLE Clément

Encadrants : LECHEVALLIER Thomas - MARTIN Jean-Yves - SERVIÈRES Myriam

Client : Parc zoologique "Planète sauvage"

Date : 24 mars 2026

Ecole Centrale de Nantes

Table des matières

1	Introduction	2
2	Contexte, enjeux et périmètre	2
3	Organisation du projet et pilotage	3
4	Évolution du backlog et pilotage opérationnel	4
5	Déroulement technique du projet	4
5.1	Mise en place de la base applicative	4
5.2	Structuration fonctionnelle et sécurité	5
5.3	Intégration IoT	5
5.4	Industrialisation de l'architecture	5
5.5	Fonctionnalités métier finalisées	6
6	Qualité logicielle, tests et documentation	6
7	Choix d'hébergement et déploiement final	7
8	Évaluation de la réussite du projet par rapport au cahier des charges	7
8.1	Éléments clairement atteints	7
8.2	Éléments partiellement atteints ou restant à consolider	7
8.3	Taux de réussite proposé	8
9	Estimation des coûts du projet	8
9.1	Méthode d'estimation	8
9.2	Heures explicitement visibles dans les rapports	8
9.3	Estimation consolidée en heures	9
9.4	Coût en heures par personne	9
9.5	Estimation du coût total en euros	9
10	Difficultés rencontrées et réponses apportées	10
11	Perspectives d'amélioration	10
12	Conclusion	11

1 Introduction

Le projet PEP'S s'inscrit dans une problématique concrète de bien-être animal au sein du parc Planète Sauvage. Son objectif est de fournir une plateforme web permettant de superviser des modules interactifs déployés sur le terrain, de gérer des contenus sonores, de collecter les interactions remontées par les dispositifs embarqués et de donner aux soigneurs et chercheurs un outil exploitable au quotidien. Le périmètre du projet SI couvre donc l'interface web, l'API backend, la gestion des données, ainsi que l'intégration avec les modules IoT de type ESP32.

Dès le départ, le projet a été pensé comme un système multi-acteurs et multi-profils. Le cahier des charges distingue notamment des administrateurs, des chercheurs/soigneurs et des modules IoT, avec une contrainte de supervision en temps réel, de gestion des rôles, de téléversement des sons, et de maintien d'une architecture suffisamment robuste pour un déploiement sur serveur externe. Les objectifs techniques fixés comprenaient aussi une forte exigence sur la qualité logicielle, la documentation, la conteneurisation Docker et la capacité à maintenir de bonnes performances d'affichage.

Au fil du projet, l'équipe a progressivement transformé une base applicative encore locale en un système beaucoup plus industrialisé : architecture Docker, proxy inverse Nginx, stockage objet, file d'attente asynchrone, authentification modernisée, montée en qualité, documentation renforcée et déploiement effectif sur VPS. Ce rapport final présente cette trajectoire de manière synthétique, en retraçant le déroulement global du projet et en évaluant le niveau d'atteinte des objectifs.

2 Contexte, enjeux et périmètre

Le besoin initial repose sur un système d'interaction animal-machine destiné à enrichir l'environnement des aras, puis à étendre l'approche à d'autres espèces comme les dauphins. Le système physique détecte une interaction, déclenche localement un son, puis envoie les données vers une plateforme web pour permettre leur exploitation. Le cahier des charges insiste sur plusieurs attentes fortes : la supervision de l'état des modules, la visualisation des interactions, la séparation des droits par profil et par espèce, la gestion des sons sans intervention technique lourde, ainsi qu'une architecture déployable hors de l'infrastructure interne du client.

Sur le plan fonctionnel, les exigences les plus structurantes étaient les suivantes :

- supervision des modules avec statut, batterie et adresse IP ;
- gestion multi-espèces ;
- téléversement et gestion de fichiers audio ;
- configuration distante et visualisation des interactions ;
- contrôle d'accès par rôles ;
- mise à jour rapide du tableau de bord.

Sur le plan technique, l'application devait rester maintenable, dockerisée et documentée, tout en gardant des performances cohérentes avec le dimensionnement visé : quelques administrateurs, quelques chercheurs/soigneurs, et un parc limité mais réel de modules ESP32.



FIGURE 1 – Cas d'utilisation synthétique du système PEP'S

3 Organisation du projet et pilotage

Le début du projet a surtout consisté à mettre en place une base de travail commune, à clarifier le cadrage et à structurer un mode de pilotage plus rigoureux. Les premiers échanges avec les encadrants ont permis de faire évoluer le cahier des charges, de mieux définir le périmètre final, d'explicitier les profils utilisateurs, de renforcer la dimension qualité et de poser des critères d'acceptation plus mesurables.

Sur le plan méthodologique, l'équipe a ensuite adopté une organisation inspirée de l'Agile/Scrum. Cette transition a apporté un cadre plus lisible pour piloter les tâches, répartir les rôles, fluidifier la validation et sécuriser les livraisons. Le projet ne s'est donc pas limité à une accumulation de développements ; il a aussi reposé sur une montée en maturité de la manière de travailler.

Un point important du déroulement a été la disponibilité irrégulière du client. Cette contrainte a eu un impact réel sur l'agilité du projet : certaines validations ont été moins rapides que prévu, plusieurs décisions fonctionnelles ont nécessité des arbitrages indirects, et l'équipe a dû parfois avancer avec un niveau d'incertitude plus élevé qu'idéal. Dans ce contexte, M. Lechevallier a joué un rôle structurant en représentant le client sur plusieurs points de cadrage, ce qui a permis de sécuriser la direction fonctionnelle de l'application et d'éviter un blocage sur les sujets d'architecture, d'intégration IoT et de priorisation.

The Team		
Team Members	Roles	Description
Haytam	Project Owner + Tester	Responsible for the final product vision, managing the backlog, and ensuring product quality through testing.
Ilyas	Scrum Master	Facilitates the Scrum team, helps remove impediments, and ensures Scrum practices are followed.
Dia	Documentation Dev & Client	Manages all project documentation and acts as the primary liaison between the team and the client.
Anas	Front-Back end	Develops and maintains both the user interface (front-end) and the server-side logic and database (back-end).
Clément	Code Quality + SonarCloud	Focuses on maintaining high standards of code quality and utilizing tools like SonarCloud for continuous inspection and improvement.
Santiago	Deployment & Security	Responsible for deploying the application and ensuring its security across all environments.

FIGURE 2 – Organisation projet et rôles dans la démarche Agile

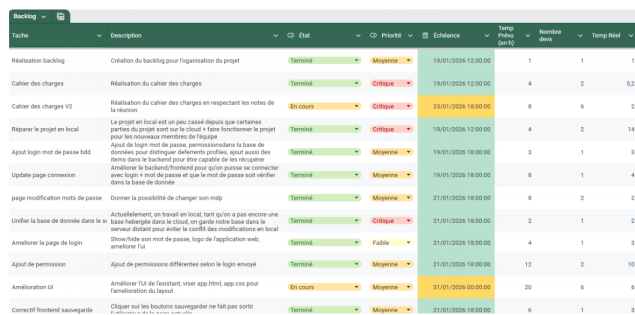
4 Évolution du backlog et pilotage opérationnel

Le pilotage opérationnel a connu deux phases distinctes.

Dans un premier temps, le backlog a été tenu dans un fichier de type tableur. Cette solution présentait un avantage très clair : elle était gratuite, simple à partager et rapide à mettre en place. Pour un projet étudiant avec une contrainte budgétaire forte, cette approche était cohérente au démarrage. Elle a permis de structurer les premières tâches, d'ajouter des échéances, de suivre les dépendances et de donner une première visibilité d'ensemble.

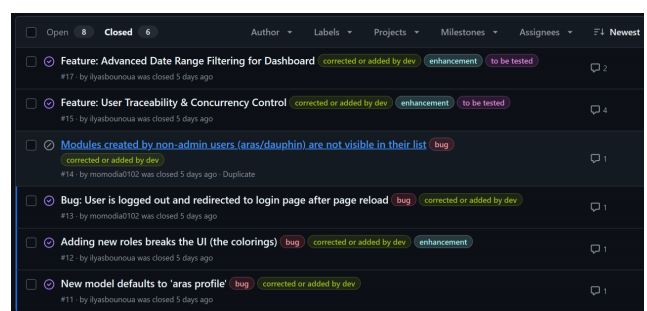
Néanmoins, cette solution a montré ses limites à mesure que le projet gagnait en profondeur technique. Le suivi d'un backlog dans un fichier Excel ou assimilé restait moins pratique dès lors qu'il fallait gérer des branches de développement, des tickets techniques détaillés, des corrections de bugs, des revues, des responsabilités précises et un lien direct avec le dépôt de code. C'est pourquoi cette approche a été abandonnée en milieu de projet au profit d'un suivi davantage centré sur les *GitHub Issues*.

Cette bascule a constitué un vrai progrès opérationnel. Les tickets sont devenus plus techniques, mieux assignés, plus faciles à faire vivre et plus proches de l'implémentation réelle. L'équipe a ainsi gagné en traçabilité, en lisibilité et en réactivité, notamment pour les sujets de qualité, d'infrastructure et d'intégration continue.



Tâche	Description	Etat	Priorité	Echéance	Temp. Prévu (h)	Nombre de tâches	Temp. Réel (h)
Réalisation backlog	Création du backlog pour l'organisation du projet	Terminé	Moyenne	19/01/2026 12:00:00	1	1	1
Cahier des charges	Réalisation du cahier des charges	Terminé	Critique	19/01/2026 12:00:00	4	2	5,2
Cahier des charges V2	Réalisation du cahier des charges en respectant les notes de la réunion	En cours	Critique	23/01/2026 18:00:00	8	6	2
Régler le projet en local	Le projet en local est en panne depuis que certaines parties du projet ont été modifiées. Sans fonctionner le projet pour les nouveaux membres de l'équipe	Terminé	Critique	19/01/2026 12:00:00	4	2	14
Ajout login mot de passe ldd	Ajout du login mot de passe, permettant d'accéder à la base de données pour distinguer différents profils, ajout aussi des notes dans le backlog pour être capable de les récupérer	Terminé	Moyenne	19/01/2026 18:00:00	3	1	3
Upgrade page connexion	Améliorer le back-end/frontend pour qu'on puisse se connecter avec login + mot de passe et que le mot de passe soit verrouillé dans la base de données	Terminé	Moyenne	19/01/2026 18:00:00	8	1	4
page modification mots de passe	Donner la possibilité de changer son mdp	Terminé	Moyenne	21/01/2026 18:00:00	8	2	2
Unifier la base de données dans le serveur distant pour éviter le conflit des modifications en local	Actuellement, on travaille en local, tant qu'on a pas encore une base hébergée dans le cloud, on garde notre base dans le serveur distant pour éviter le conflit des modifications en local	Terminé	Critique	21/01/2026 18:00:00	2	1	2
Améliorer la page de login	Bluebird ne veut pas de passe, logo de l'application web, améliorer l'ui	Terminé	Faible	21/01/2026 18:00:00	4	1	3
Ajout de permission	Ajout de permissions différentes selon le login envoyé	Terminé	Moyenne	21/01/2026 18:00:00	12	2	10
Amélioration UI	Améliorer l'UI de l'assistant, vision app html, app css pour l'amélioration du layout	En cours	Moyenne	31/01/2026 00:00:00	20	6	6
Correctif frontend sauvegarde	Clicker sur les boutons sauvegarder ne fait pas sortir l'application de la page	Terminé	Moyenne	21/01/2026 18:00:00	6	1	3

(a) Suivi initial du travail collaboratif



(b) Pilotage technique via GitHub Issues

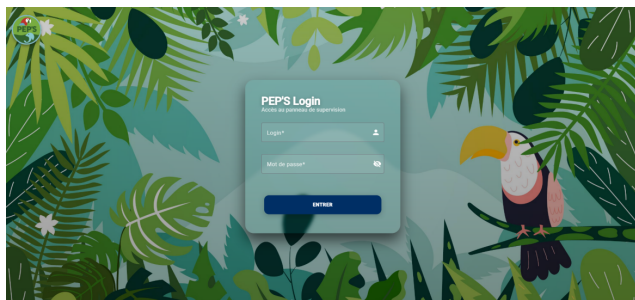
FIGURE 3 – Évolution des outils de pilotage du backlog

5 Déroulement technique du projet

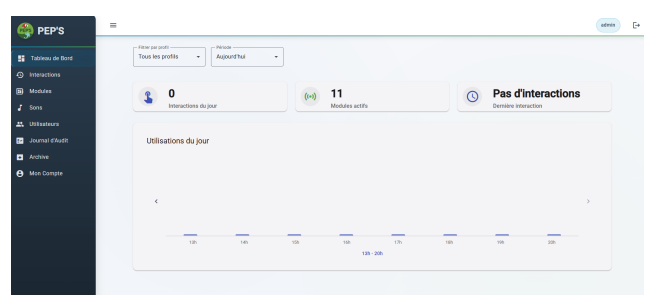
5.1 Mise en place de la base applicative

Les premières étapes ont consisté à consolider le socle du système. Une authentification sécurisée a été mise en place côté backend avec stockage PostgreSQL et chiffrement des mots de passe. En parallèle, la base de données a été centralisée sur un serveur distant afin de faciliter le travail collaboratif et d'éviter les divergences entre environnements locaux.

Côté interface, une première refonte de la page de connexion puis de l'interface principale a amélioré la lisibilité du produit, l'ergonomie générale et la cohérence visuelle. Cette phase a été importante car elle a permis d'installer une base réutilisable, plus propre et plus stable, avant de complexifier l'application.



(a) Page de connexion



(b) Interface principale

FIGURE 4 – Premiers écrans stabilisés du système

5.2 Structuration fonctionnelle et sécurité

Une fois cette base posée, l'équipe a travaillé sur les fonctions métier les plus utiles au cahier des charges :

- journalisation des actions ;
- gestion des rôles et permissions ;
- verrouillage optimiste pour éviter les conflits d'écriture ;
- prise en charge du multi-profils et du multi-espèces ;
- sécurisation progressive de la navigation.

Le projet a ensuite franchi une étape importante avec la refonte de l'authentification vers JWT et l'utilisation de *Guards* Angular pour renforcer les contrôles d'accès. Cette évolution a allégé le fonctionnement du système, amélioré la sécurité applicative et rendu le routage plus propre.

5.3 Intégration IoT

L'intégration avec le système embarqué a constitué l'un des fils rouges du projet. Il a fallu préparer des endpoints compatibles avec les modules, assouplir certains mécanismes de sécurité sur les routes dédiées, simuler les échanges puis valider la remontée des interactions depuis le terrain.

Une difficulté non négligeable est apparue lorsque l'architecture matérielle côté IoT a évolué d'un modèle fondé sur Raspberry Pi vers un ensemble de cartes ESP32. Heureusement, cette modification n'a pas remis en cause la logique générale du projet SI. Le protocole d'échange a été maintenu, et le backend a pu absorber cette évolution sans rupture majeure.

La robustesse de la connectivité a ensuite été renforcée avec un mécanisme de configuration dynamique du réseau. Cela répond à une vraie contrainte d'usage : si le réseau local du client change, il devient possible de reconfigurer les cartes sans opération lourde de maintenance. Cette avancée a fortement réduit le risque opérationnel du système après livraison.

5.4 Industrialisation de l'architecture

L'un des changements les plus structurants du projet a été l'industrialisation progressive de l'architecture.

L'application a d'abord été entièrement dockerisée afin de standardiser son exécution et de préparer un déploiement reproductible. Un proxy inverse Nginx a ensuite été placé en point d'entrée unique pour centraliser le trafic, mieux contrôler l'exposition des services et préparer un fonctionnement plus proche d'une architecture de production.

La pile a ensuite été enrichie avec :

- PostgreSQL pour la persistance des données ;
- MinIO pour le stockage objet des fichiers audio ;
- Redis, puis Redis Streams, pour la gestion de traitements asynchrones ;
- des workers dédiés au traitement et au stockage.

Cette architecture a aussi permis d'introduire une logique de réplicas backend et de répartition de charge via Nginx. Le projet est ainsi passé d'une logique de prototype applicatif à une logique bien plus proche d'un système exploitable.

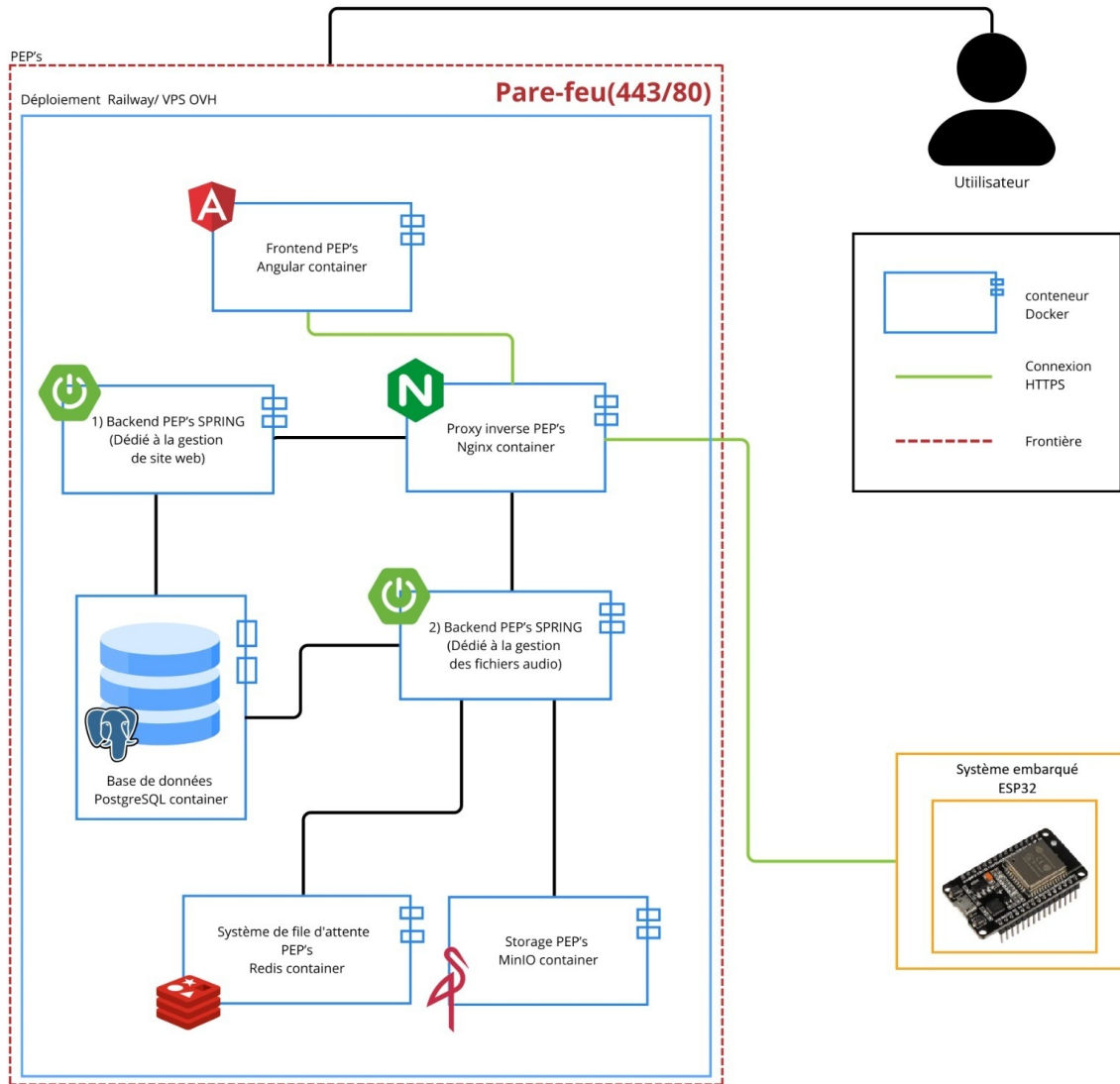


FIGURE 5 – Architecture cible industrialisée avec Nginx, MinIO et Redis

5.5 Fonctionnalités métier finalisées

En fin de projet, plusieurs fonctionnalités importantes ont été consolidées :

- gestion des fichiers audio et de leur association aux modules ;
- validation des formats et contraintes de téléversement ;
- système d'alertes pour les événements critiques ;
- page de gestion du profil utilisateur ;
- finalisation du flux complet IoT → backend → base de données ;
- amélioration de la réactivité grâce au pipeline asynchrone.

L'ensemble dessine une application cohérente avec le besoin exprimé : superviser, configurer, historiser et exploiter des interactions remontées depuis des modules déployés dans un environnement réel.

6 Qualité logicielle, tests et documentation

La qualité n'a pas été traitée comme une activité annexe. Elle a accompagné le projet tout au long de son déroulement.

Dès les premières itérations, l'équipe a structuré une logique de tests distincte du développement. Un workflow de validation a été mis en place pour éviter qu'un membre teste seul ses propres développements. Parallèlement, des inventaires de tests fonctionnels ont été produits afin de mieux couvrir les fonctions critiques de l'application.

Sur la partie backend puis frontend, un travail régulier a aussi été mené pour améliorer la couverture de code et traiter les alertes remontées par Sonar. Les rapports font apparaître une progression nette du niveau de qualité, en particulier sur la partie frontend, avec un gain important sur le coverage. La documentation a été enrichie progressivement, à la fois sur la structure logicielle, les conventions, les procédures de déploiement et les composants de l'architecture.

Ce travail de fond est important dans le bilan final, car il explique pourquoi le projet ne s'arrête pas à une démonstration fonctionnelle. Il aboutit à un produit techniquement plus reprenable, plus robuste et plus crédible dans une logique de continuité.

7 Choix d'hébergement et déploiement final

Le sujet du déploiement a été étudié très tôt, avec une comparaison de plusieurs approches et de plusieurs fournisseurs. Au départ, plusieurs options étaient envisageables : PaaS, CaaS, VPS ou déploiement plus classique. Au fil du projet, le choix d'un VPS OVH s'est imposé comme la solution la plus adaptée.

Ce choix a été motivé par plusieurs arguments :

- un bon rapport qualité/prix ;
- la liberté de configuration nécessaire à une architecture Docker + Nginx + Spring + Angular + Redis + MinIO ;
- une localisation française des datacenters ;
- l'impossibilité d'intégrer directement la solution sur l'infrastructure du client, gérée par un prestataire externe.

Un point majeur à retenir est que le déploiement sur VPS est désormais réalisé au 13 mars 2026. Cette mise en production a été rendue possible parce que l'architecture avait été préparée en amont dans cette perspective : conteneurisation, séparation des services, variables d'environnement, proxy inverse, optimisation progressive de l'infrastructure. Autrement dit, le déploiement final n'a pas été une improvisation de dernière minute, mais l'aboutissement logique d'un travail d'industrialisation mené plusieurs semaines auparavant.

L'application est actuellement accessible sur une adresse IP publique :

<http://51.75.126.85/login>

Cette disponibilité effective constitue une avancée importante du projet, même si elle reste encore perfectible tant qu'aucun nom de domaine n'est associé à l'instance.

8 Évaluation de la réussite du projet par rapport au cahier des charges

L'évaluation du taux de réussite doit rester nuancée. Le projet a clairement atteint une grande partie de ses objectifs structurants, mais tout n'est pas clos avec le même niveau de maturité.

8.1 Éléments clairement atteints

Les points suivants peuvent être considérés comme atteints ou très majoritairement atteints :

- architecture web opérationnelle ;
- gestion multi-profil et contrôle d'accès par rôles ;
- téléversement et gestion des sons ;
- intégration des échanges IoT/backend ;
- supervision et remontée des données ;
- dockerisation de l'application ;
- choix d'hébergement cohérent puis déploiement effectif sur VPS ;
- documentation technique significativement enrichie ;
- nette progression de la qualité logicielle.

8.2 Éléments partiellement atteints ou restant à consolider

Quelques sujets restent plus partiels :

- accessibilité numérique encore en remédiation ;

- finition UI encore améliorable ;
- certains critères de performance et de recette client non démontrés de manière exhaustive dans le cadre du rapport final ;
- absence actuelle de domaine dédié.

8.3 Taux de réussite proposé

En retenant le cahier des charges comme référence principale, et en distinguant les fonctions livrées, les briques d'infrastructure, la qualité et les sujets encore partiellement ouverts, le taux de réussite du projet peut être estimé à **environ 85 %**.

Cette valeur nous paraît défendable pour les raisons suivantes :

- les fonctions cur du produit sont présentes ;
- l'intégration technique la plus risquée a été menée à terme ;
- le déploiement réel a été effectué ;
- les écarts restants portent davantage sur la finition, l'accessibilité, certaines validations formelles et les perspectives post-projet que sur le cur fonctionnel.

Ce taux traduit donc un projet globalement réussi, techniquement sérieux, mais avec encore une marge d'amélioration normale pour passer d'un très bon prototype industrialisé à un produit totalement finalisé.

9 Estimation des coûts du projet

9.1 Méthode d'estimation

Les rapports d'avancement ne fournissent pas un relevé exhaustif des heures réellement pointées par chaque membre. En revanche, ils donnent plusieurs charges estimées sur des tâches d'infrastructure, ainsi qu'un effort identifié de remédiation accessibilité. Pour produire une estimation globale cohérente, nous proposons donc une **reconstitution raisonnable** du coût du projet, fondée sur :

- les estimations explicites présentes dans le backlog technique ;
- l'ampleur des chantiers effectivement réalisés ;
- la durée globale du projet ;
- la taille de l'équipe (6 étudiants) ;
- la charge réellement observée, d'environ 50 heures par étudiant.

Nous distinguons ainsi :

1. un **noyau d'heures explicitement visibles** dans les rapports ;
2. une **estimation consolidée** de l'ensemble du travail projet.

9.2 Heures explicitement visibles dans les rapports

TABLE 1 – Charges explicites retrouvées dans les rapports

Réf.	Description	Charge estimée
INFRA-001	Mise en uvre de MinIO (service, SDK, contrôleur)	12 à 13 h
INFRA-002	Implémentation de Redis comme file d'attente	10 à 12 h
INFRA-003	Configuration des répliques backend	2 à 3 h
TOOL-001	Script de migration vers MinIO	3 à 5 h
INFRA-004	Optimisation Nginx (cache, compression)	3 à 5 h
UI-001	Remédiation accessibilité	8 h
Total explicite		38 à 46 h

Ce total ne couvre cependant qu'une partie du projet. Il manque notamment :

- la conception initiale ;
- le cadrage et la mise à jour du cahier des charges ;

- la mise en place de l'authentification et des rôles ;
- la refonte UI ;
- les tests, la QA et SonarCloud ;
- la documentation ;
- l'intégration IoT ;
- les arbitrages de déploiement ;
- les réunions de coordination.

9.3 Estimation consolidée en heures

Au vu de l'ensemble des livrables réalisés et de la charge réellement constatée au sein du groupe, une estimation consolidée plus fidèle du projet est la suivante :

TABLE 2 – Estimation consolidée de la charge projet

Poste	Heures estimées
Cadrage, réunions, backlog, pilotage	40 h
Conception fonctionnelle et architecture	30 h
Développement backend	58 h
Développement frontend	55 h
Intégration IoT et tests réseau	32 h
Infrastructure, Docker, Nginx, VPS, MinIO, Redis	38 h
Qualité, tests, SonarCloud, corrections	25 h
Documentation technique et reporting	22 h
Total groupe estimé	300 h

Cette estimation de **300 heures** paraît cohérente avec :

- la charge réellement fournie par l'équipe ;
- le niveau de complexité du système ;
- la multiplicité des chantiers menés en parallèle ;
- l'existence d'un vrai travail d'industrialisation, et pas seulement d'une maquette.

9.4 Coût en heures par personne

Avec une équipe de 6 personnes, la charge moyenne par membre est :

$$\frac{300}{6} = 50 \text{ heures par personne}$$

On peut donc retenir :

- **coût total groupe : 300 h**
- **coût moyen par personne : 50 h**

9.5 Estimation du coût total en euros

Pour convertir ces heures en euros, il faut poser une hypothèse. Dans un cadre académique, une hypothèse simple et lisible consiste à utiliser un **coût moyen valorisé à 25 /h**. Cette valeur ne prétend pas refléter un tarif de marché senior, mais plutôt une valorisation raisonnable du travail d'ingénierie réalisé dans le cadre d'un projet de fin d'étude.

Le coût total valorisé devient alors :

$$300 \times 25 = 7\,500$$

Nous retenons donc :

TABLE 3 – Synthèse des coûts

Indicateur	Valeur
Charge totale groupe	300 h
Charge moyenne par personne	50 h
Taux horaire retenu	25 /h
Coût total estimé du projet	7 500

Remarque : avec une valorisation plus prudente à 20 /h, le coût serait de 6 000 ; à 30 /h, il serait de 9 000 . Pour un rendu académique, 25 /h constitue un bon compromis entre crédibilité et lisibilité.

10 Difficultés rencontrées et réponses apportées

Le projet a été confronté à plusieurs difficultés, qui ont influencé son déroulement sans remettre en cause sa réussite globale.

La première difficulté a concerné la disponibilité du client. Cette situation a rendu certains retours plus espacés et a réduit la fluidité d'un pilotage agile idéal. Pour compenser cela, les encadrants, et en particulier M. Lechevallier, ont joué un rôle important dans la stabilisation du cadrage et dans l'orientation du produit sur les points les plus sensibles.

La deuxième difficulté a été liée à l'intégration IoT. Le changement d'architecture côté matériel, les contraintes de terrain et les problèmes de connexion réseau ont demandé plusieurs adaptations, notamment pour les tests en réseau local et la robustesse de la reconnexion.

La troisième difficulté a été technique et collaborative : conflits de fusion, nécessité de maintenir une branche principale stable, montée en complexité de l'architecture et besoin d'outils de pilotage plus adaptés que le backlog initial sous tableur.

Enfin, la question de l'hébergement a demandé un arbitrage autonome de l'équipe, car la solution ne pouvait pas être intégrée directement aux serveurs du client. Ce point a finalement débouché sur un choix clair, cohérent et maîtrisé : le VPS OVH.

11 Perspectives d'amélioration

Le projet constitue une base fonctionnelle solide, mais plusieurs améliorations pourraient être envisagées par une équipe future afin d'enrichir le système et d'améliorer son exploitation :

- **Amélioration du contrôle des modules IoT :** mettre en place un système de supervision plus avancé permettant de suivre l'état des modules (connectivité, statut, niveau de batterie) et d'envoyer des notifications automatiques lorsque les modules deviennent inactifs ou lorsque la batterie est faible.
- **Intégration d'une couche d'intelligence artificielle :** développer un mécanisme adaptatif capable d'ajuster automatiquement les paramètres sonores (volume, fréquence, type de son) en fonction du comportement observé des animaux et de l'historique des interactions.
- **Amélioration de l'interface utilisateur :** refondre certains écrans pour améliorer l'ergonomie, la visualisation des données et l'accessibilité pour les soigneurs et les chercheurs.
- **Ajout d'un nom de domaine et sécurisation HTTPS :** associer l'application à un domaine dédié et configurer une couche TLS complète afin d'améliorer la sécurité et la lisibilité du système.
- **Renforcement des capacités d'analyse des données :** développer des tableaux de bord plus avancés permettant d'analyser les interactions, de comparer les comportements entre espèces et d'exploiter plus finement les données collectées.
- **Amélioration de la maintenabilité technique :** poursuivre l'amélioration de la documentation technique, automatiser davantage les tests et mettre en place des pipelines d'intégration continue afin de faciliter la reprise et l'évolution du projet par les équipes futures.

12 Conclusion

Le projet PEP'S a évolué d'une base applicative encore en structuration vers une plateforme web cohérente, dockerisée, intégrée au système IoT, qualitativement mieux maîtrisée et effectivement déployée sur VPS. Cette progression ne s'est pas faite uniquement par accumulation de fonctionnalités ; elle s'est surtout construite par paliers de maturité successifs : clarification du besoin, structuration du pilotage, amélioration du backlog, montée en exigence qualité, industrialisation de l'architecture et sécurisation du déploiement.

Malgré un contexte parfois moins favorable qu'attendu, notamment du fait d'une disponibilité limitée du client et de contraintes techniques côté terrain, l'équipe a su maintenir un cap clair. Le rôle d'appui joué par M. Lechevallier dans le cadrage a été déterminant pour préserver cette continuité. De la même façon, la transition d'un backlog simple vers un pilotage par *GitHub Issues*, puis le choix d'une architecture conteneurisée prête à être hébergée, montrent que le projet a aussi progressé dans sa manière d'être mené.

Au regard du cahier des charges, le résultat est globalement très satisfaisant. Les briques cur sont en place, le produit existe réellement, l'intégration technique la plus risquée a été menée à terme, et le déploiement est effectif au 13 mars 2026. Le projet peut ainsi être considéré comme une réussite solide, avec un taux d'aboutissement estimé à environ 85 %, tout en conservant des pistes d'évolution crédibles pour une suite future plus ambitieuse.